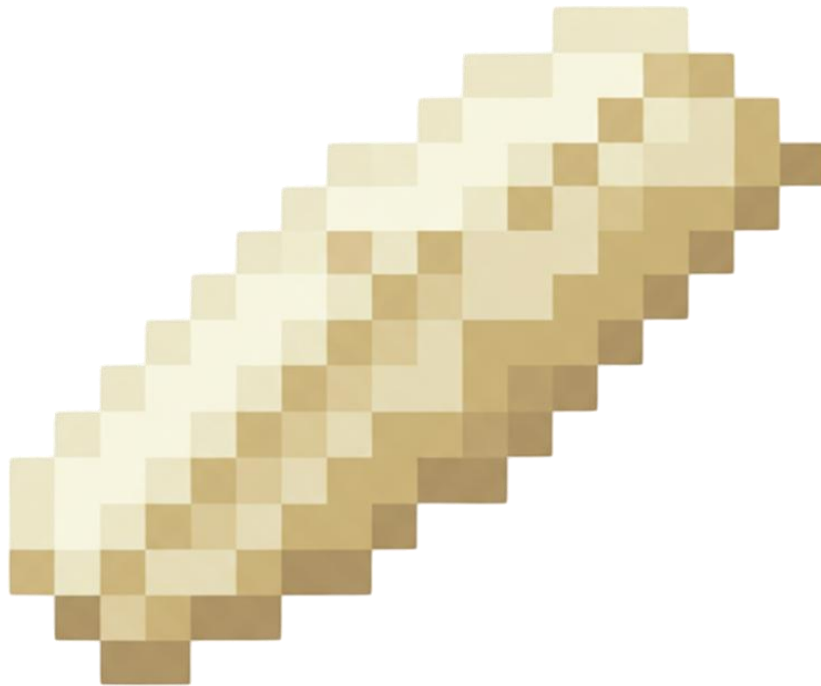


PRÁCTICA 1:

ARQUITECTURA

WEB



- Juan Manuel Valenzuela González
- Artur Vargas Carrión
- Jorge Repullo Serrano
- Eduardo González Bautista
- Rubén Oliva Zamora
- David Muñoz del Valle

`amcgil@uma.es`

`arturvargas797@uma.es`

`jorgers4@uma.es`

`edugb@uma.es`

`rubenoliva@uma.es`

`davidmunvalle@uma.es`



ÍNDICE DE CONTENIDO

1. DESCRIPCIÓN DE LA APLICACIÓN - BASMATI.....	1
1.1. ¿QUÉ ES <i>BASMATI</i> ?.....	1
1.2. ¿POR QUÉ EL NOMBRE <i>BASMATI</i> ?.....	1
1.3. ¿CÓMO SE LLEVARÁ A CABO?.....	1
1.4. REQUISITOS.....	2
1.4.1. <i>Requisitos funcionales</i>	2
1.4.1.1. Gestión de calendarios	2
1.4.1.2. Gestión de eventos	2
1.4.1.3. Visualización y búsqueda.....	2
1.4.1.4. Gestión de usuarios y autenticación	3
1.4.1.5. Interactividad y notificaciones.....	3
1.4.1.6. Integración con servicios externos.....	3
1.4.2. <i>Requisitos no funcionales</i>	3
2. ARQUITECTURA DE LA APLICACIÓN.....	5
2.1. DIAGRAMA DE DESPLIEGUE	5
2.1.1. <i>Cliente</i>	6
2.1.2. <i>Cliente web</i>	6
2.1.3. <i>Servidor de aplicaciones</i>	6
2.1.4. <i>Sistemas externos</i>	6
2.1.4.1. Almacenamiento de archivos (GCS).....	6
2.1.4.2. Servidor de autenticación (OAuth 2.0).....	7
2.1.4.3. Otros servicios externos	7
2.2. DIAGRAMA DE COMPONENTES	8
2.2.1. <i>Cliente</i>	9
2.2.2. <i>Backend</i>	9
2.2.3. <i>Microservicios internos</i>	9
2.2.4. <i>APIs externas</i>	10
2.2.5. <i>Base de datos</i>	11
3. DESCRIPCIÓN DE TECNOLOGÍAS ELEGIDAS	12
3.1. BACKEND (PYTHON)	12
3.1.1. <i>Riesgos asociados</i>	12
3.1.2. <i>Alternativas consideradas</i>	12
3.2. FRONTEND (REACT + VITE)	12
3.2.1. <i>Riesgos asociados</i>	13
3.2.2. <i>Alternativas consideradas</i>	13
3.3. API (FASTAPI).....	13
3.3.1. <i>Riesgos asociados</i>	13
3.3.2. <i>Alternativas consideradas</i>	13
3.4. BASE DE DATOS (MONGODB).....	14
3.4.1. <i>Riesgos asociados</i>	14
3.4.2. <i>Alternativas consideradas</i>	14
3.5. CLOUD (GOOGLE CLOUD)	14
3.5.1. <i>Riesgos asociados</i>	15
3.5.2. <i>Alternativas consideradas</i>	15
3.6. SISTEMA DE ALMACENAMIENTO DE ARCHIVOS (MONGODB + GCS)	15
3.6.1. <i>Riesgos asociados</i>	15
3.6.2. <i>Alternativas consideradas</i>	16

1. Descripción de la aplicación - Basmati

1.1. ¿Qué es *Basmati*?

Basmati, es una aplicación que deriva de la aplicación propuesta Kalendas la cual tiene un funcionamiento similar a *Teamup*. Se trata de una aplicación web que **gestiona calendarios de eventos**. Esta aplicación incluirá diversas funcionalidades, tales como la creación, edición y mantenimiento de calendarios de eventos y sus contenidos asociados.

1.2. ¿Por qué el nombre *Basmati*?

El nombre *Basmati* surge como una metáfora de la vida misma. Al igual que un saco de arroz, nuestra vida está compuesta por innumerables momentos, tareas y responsabilidades que, en conjunto, pueden parecer caóticos y difíciles de organizar. Tratar de poner orden a todo ello puede sentirse tan complejo como intentar **organizar los granos de arroz uno por uno**. Basmati nace con el propósito de simplificar este proceso: ordenar, estructurar y dar claridad a ese aparente desorden, para que lo realmente importante no sea gestionar el caos, sino disfrutar plenamente de la vida.

1.3. ¿Cómo se llevará a cabo?

Para la realización de las partes que componen la aplicación, usaremos diversas tecnologías:

- **Backend:** Python, debido a la facilidad de aprendizaje de este lenguaje y su fácil integración con herramientas de IA.
- **Frontend:** React (junto con Vite) es una muy buena opción por su orden y gestión de estado, además de que ya hemos trabajado con esta tecnología.
- **API:** FastAPI debido a su velocidad y su sencilla curva de aprendizaje.
- **Base de datos:** MongoDB, pues tiene una fácil integración con Python y es muy simple y escalable.
- **Cloud:** Google Cloud, que nos permite trabajar con estructuras dinámicas y consultar datos en tiempo real de manera eficiente.
- **Sistema de almacenamiento de archivos:** utilizaremos una arquitectura combinada entre MongoDB y Google Cloud Storage (GCS). Los archivos de gran tamaño se almacenarán directamente en GCS, mientras que en MongoDB se guardarán únicamente las URL de acceso correspondientes y los archivos más livianos.

1.4. Requisitos

1.4.1. Requisitos funcionales

1.4.1.1. Gestión de calendarios

RF1. Creación de calendarios (usuario registrado): el sistema debe permitir la creación de calendarios temáticos, identificados por un título, un organizador y palabras clave.

RF2. Edición de calendarios (creador): el sistema debe permitir la modificación de la información de un calendario únicamente por parte de su creador.

RF3. Eliminación de calendarios (creador): el sistema debe permitir la eliminación de un calendario únicamente por parte de su creador.

RF4. Relaciones jerárquicas (usuario registrado): el sistema debe permitir establecer relaciones jerárquicas entre calendarios (ejemplo: un calendario de curso que agrupe los calendarios de sus asignaturas).

1.4.1.2. Gestión de eventos

RF5. Creación de eventos (creador del calendario): el sistema debe permitir añadir eventos dentro de un calendario. Cada evento debe contener al menos: título, hora de inicio y duración. Opcionalmente se podrá añadir, entre otras cosas, lugar de celebración.

RF6. Modificación de eventos (creador del calendario): el sistema debe permitir la modificación de eventos únicamente por parte del creador del calendario asociado.

RF7. Eliminación de eventos (creador del calendario): el sistema debe permitir la eliminación de eventos únicamente por parte del creador del calendario asociado.

RF8. Enriquecimiento de eventos (creador del calendario): el sistema debe permitir añadir contenido adicional a los eventos, como imágenes, archivos adjuntos, comentarios o mapas.

1.4.1.3. Visualización y búsqueda

RF9. Visualización de calendarios (todos los usuarios): el sistema debe permitir que cualquier usuario, registrado o no, pueda visualizar los calendarios creados que tenga permitido ver.

RF10. Búsqueda de calendarios (todos los usuarios): el sistema debe permitir la búsqueda de calendarios por criterios como título, organizador o palabras clave.

RF11. Vista unificada de calendarios (usuario registrado): el sistema debe ofrecer una vista agregada que muestre todos los calendarios y subcalendarios que un usuario decida seguir o visualizar.

1.4.1.4. Gestión de usuarios y autenticación

RF12. Autenticación mediante cuentas externas (todos los usuarios): el acceso para la creación y gestión de contenido se realizará utilizando un sistema de cuentas externas (Google, Facebook) bajo el protocolo OAuth 2.0.

RF13. Permisos de gestión de calendarios y eventos (creador): solo el usuario que ha creado un calendario tendrá permisos para modificarlo y gestionar sus eventos asociados.

1.4.1.5. Interactividad y notificaciones

RF14. Comentarios en eventos (todos los usuarios): el sistema debe permitir que los usuarios añadan comentarios en los eventos.

RF15. Notificación de comentarios (creador del evento): el sistema debe notificar al creador de un evento cuando se añada un nuevo comentario.

RF16. Configuración de notificaciones (usuario registrado): el sistema debe permitir a los usuarios decidir si quieren recibir notificaciones por correo electrónico o visualizarlas al iniciar sesión en la aplicación.

1.4.1.6. Integración con servicios externos

RF17. Importación de calendarios (usuario registrado): el sistema debe permitir la importación de calendarios mediante APIs de servicios web externos o conjuntos de datos abiertos (ejemplo: API de Teamup).

RF18. Enriquecimiento con datos externos (usuario registrado): el sistema debe permitir el uso de servicios externos para complementar la información de calendarios o eventos.

1.4.2. Requisitos no funcionales

RNF1. Base de datos no relacional: el sistema debe utilizar una base de datos no relacional para almacenar de forma persistente la información de calendarios y eventos.

RNF2. Almacenamiento de archivos: las imágenes y archivos adjuntos se alojarán en el sistema de almacenamiento de la aplicación o en servicios externos configurables.

RNF3. Accesibilidad web: la aplicación debe ser accesible a través de un navegador web estándar.

RNF4. Usabilidad de la interfaz: la interfaz debe ser intuitiva y sencilla para facilitar la gestión de calendarios y eventos.

RNF5. Rendimiento y optimización: el sistema debe optimizar las llamadas a servicios externos (APIs, almacenamiento, etc.) para minimizar costes y respetar los límites de uso.

RNF6. Disponibilidad en la nube: la aplicación debe desplegarse en Google Cloud para garantizar su disponibilidad y acceso a través de la web.

2. Arquitectura de la aplicación

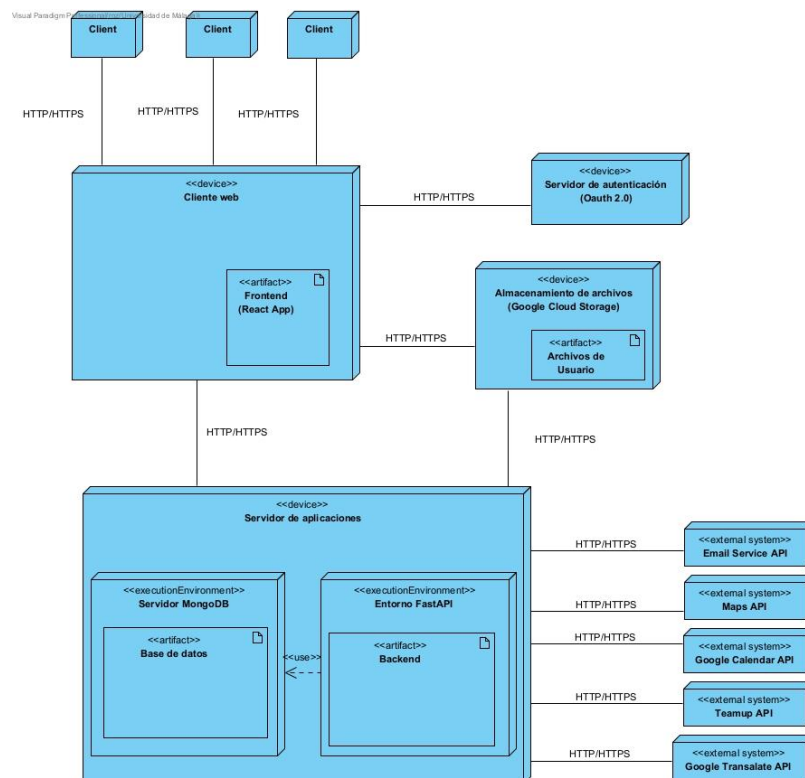
2.1. Diagrama de despliegue

El siguiente diagrama de despliegue ilustra la distribución física de la arquitectura propuesta en la etapa inicial del proyecto. Su objetivo es mostrar cómo se asignan los diferentes artefactos de software a los nodos de hardware y entornos de ejecución, así como definir los protocolos de comunicación entre ellos.

La infraestructura se basa en un modelo distribuido donde los usuarios (Client) acceden a través de protocolos HTTP/HTTPS al Cliente Web. En este dispositivo se ejecuta el artefacto Frontend, que actúa como la interfaz principal y se comunica mediante llamadas API con el servidor de aplicaciones. Este servidor central aloja dos entornos de ejecución críticos: el entorno FastAPI, que contiene el artefacto Backend con la lógica de la aplicación, y el servidor MongoDB (ejecutados en máquinas virtuales diferentes dentro de la misma máquina física), que gestiona la base de datos y es utilizado directamente por el backend para la persistencia de datos estructurados.

Además, el sistema se integra con una serie de servicios externos e infraestructura en la nube para complementar sus funcionalidades. La autenticación se gestiona a través de un servidor de autenticación (OAuth 2.0) contactado desde el cliente. Por su parte, el backend interactúa vía HTTP/HTTPS con Google Cloud Storage para el almacenamiento y recuperación de archivos de gran tamaño, y realiza llamadas API a Google Translate API para servicios de traducción.

Este esquema establece la base operativa y la topología de red sobre la cual se desplegará la solución, manteniendo la flexibilidad necesaria para evolucionar ante futuros requisitos de escalabilidad o cambios en los servicios de terceros.



A continuación, se describe con mayor detalle cada uno de los nodos y artefactos que componen el diagrama de despliegue:

2.1.1. Cliente

Representa el dispositivo físico desde el cual los usuarios finales acceden a la aplicación. Puede ser cualquier dispositivo con un navegador web, como un ordenador de sobremesa, un portátil, un teléfono móvil o incluso un agente de inteligencia artificial. Es el punto de partida de toda interacción y se comunica con el sistema a través del protocolo **HTTP/HTTPS**.

2.1.2. Cliente web

- **Cliente web (device)**: este nodo representa el entorno donde se sirve y ejecuta la interfaz de usuario. En un entorno de producción, sería un servidor web (como Nginx o Apache) o un servicio de hosting para aplicaciones.
- **Frontend (artifact)**: es el software que se ejecuta en el cliente web. Se trata de una aplicación de página única (SPA) desarrollada en React. Su responsabilidad es renderizar la interfaz, gestionar las interacciones del usuario y comunicarse con los servicios de backend y autenticación.

2.1.3. Servidor de aplicaciones

- **Servidor de aplicaciones (device)**: es el nodo central de la infraestructura y aloja la lógica de negocio principal de Basmati. Este servidor físico contiene dos entornos de ejecución diferenciados, cada uno siendo ejecutado en una máquina virtual distinta:
 - **Entorno FastAPI (executionEnvironment)**: el entorno de ejecución donde opera la lógica de la aplicación.
 - **Backend (artifact)**: el artefacto de software que contiene toda la lógica de negocio, la gestión de la API y la comunicación con la base de datos y los servicios externos.
 - **Servidor MongoDB (executionEnvironment)**: el entorno que ejecuta el sistema de gestión de bases de datos.
 - **Base de datos (artifact)**: el conjunto de datos persistentes de la aplicación (usuarios, calendarios, eventos, etc.). El Backend interactúa directamente con este artefacto para leer y escribir información, como indica la relación <<use>>.

2.1.4. Sistemas externos

2.1.4.1. Almacenamiento de archivos (GCS)

- **Almacenamiento de archivos**: representa el servicio externo dedicado al almacenamiento de ficheros de gran tamaño, que se estará ejecutando en alguna máquina que esté bajo el dominio de Google.
 - **Archivos de usuario (artifact)**: son los ficheros como imágenes, documentos adjuntos o cualquier otro contenido multimedia subido por los usuarios. El backend gestiona las

referencias (URLs) a estos archivos, mientras que el cliente web puede interactuar con GCS para subirlos o descargarlos directamente

2.1.4.2. Servidor de autenticación (OAuth 2.0)

- **Servidor de autenticación:** simboliza un servicio de identidad y autenticación externo, ejecutado en alguna máquina fuera de nuestro dominio. Gestiona el inicio de sesión y la validación de la identidad del usuario a través de proveedores como Google o Facebook. El cliente web se comunica con este servidor para obtener un token de autenticación que luego se utiliza para realizar peticiones seguras al servidor de aplicaciones.

2.1.4.3. Otros servicios externos

Entre el resto de los servicios usados existen diversas APIs de terceros que enriquecen las funcionalidades de la aplicación. El Backend se comunica con ellos vía **HTTP/HTTPS** para:

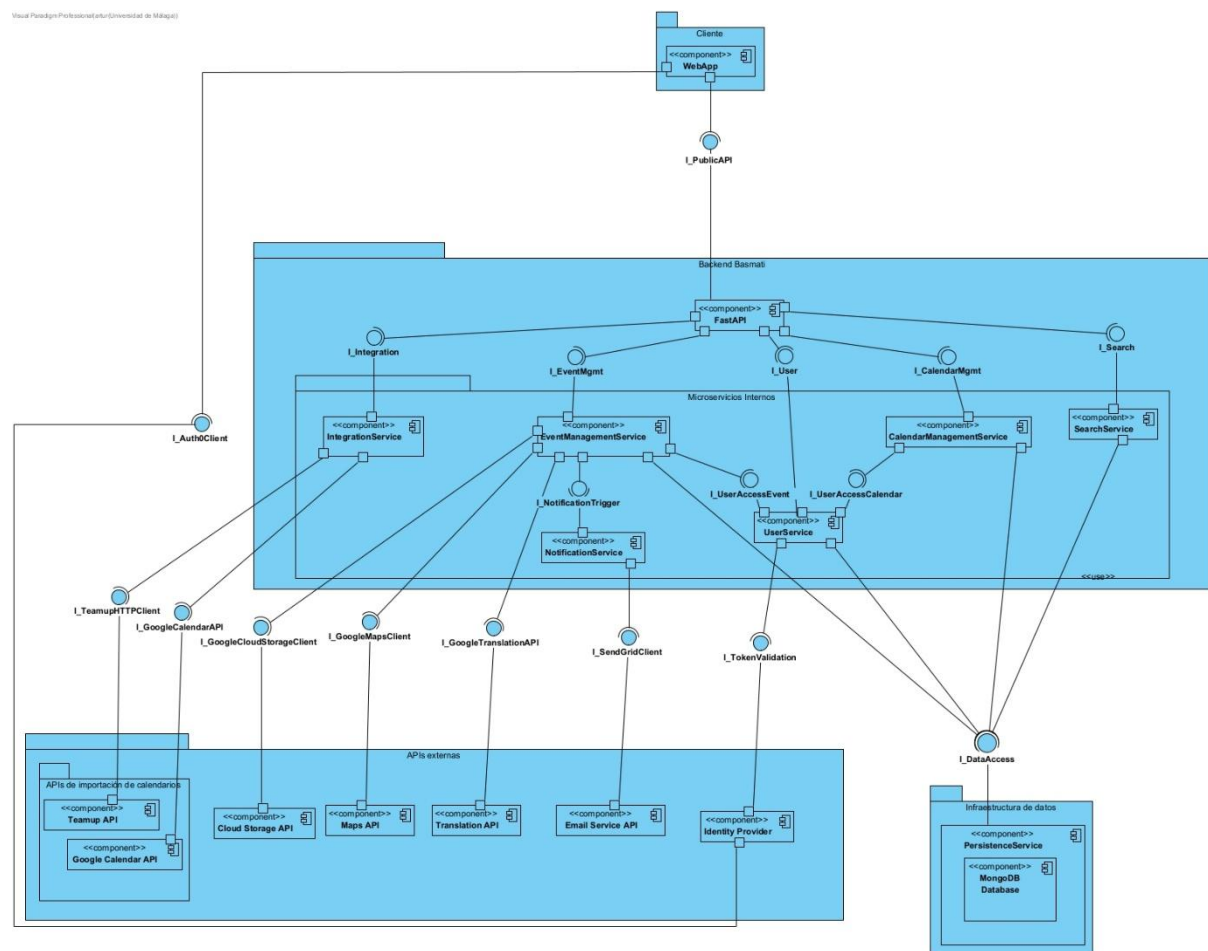
- **Email Service API:** enviar notificaciones por correo electrónico.
- **Maps API:** obtener datos de geolocalización para los eventos.
- **Google Calendar API / Teamup API:** importar calendarios y eventos de otras plataformas.
- **Google Translate API:** ofrecer funcionalidades de traducción.

2.2. Diagrama de componentes

El siguiente diagrama de componentes representa la arquitectura propuesta para el sistema Basmati. Su objetivo es ofrecer una visión global de la estructura, sus principales módulos y cómo interactúan entre sí y con componentes externos.

La aplicación se basa en una arquitectura de **cliente-servidor con microservicios**. El cliente web (WebApp) actúa como interfaz de usuario y se comunica con el backend a través de un **API Gateway**. Este backend está compuesto por distintos microservicios especializados, cada uno responsable de un conjunto concreto de funcionalidades (gestión de calendarios, eventos, búsqueda, usuarios, etc.). Esta modularidad facilita el desarrollo, despliegue y escalabilidad de la aplicación.

Además, el sistema interactúa con una infraestructura de datos basada en **MongoDB** y con diversos **servicios externos** (proveedores de identidad, almacenamiento en la nube, APIs de mapas...) para ampliar sus capacidades. El diagrama establece una base conceptual sólida sobre la cual evolucionará la solución.



A continuación, se describe con mayor detalle cada uno de los elementos del diagrama:

2.2.1. Cliente

- **<<Componente UI>> WebApp:** es el punto de entrada para el usuario. Se trata de una aplicación de página única (SPA - Single Page Application) desarrollada en React. Es responsable de toda la lógica de presentación, la interacción con el usuario y la comunicación con el backend. Consume la API pública expuesta por el API Gateway.

2.2.2. Backend

El backend agrupa toda la lógica de negocio y está diseñado siguiendo un patrón de microservicios.

- **<<API Gateway>> (FastAPI):** actúa como la única puerta de entrada (single entry point) para todas las peticiones del cliente. Sus responsabilidades son:
 - **Enrutamiento:** dirigir cada petición entrante al microservicio interno correspondiente.
 - **Autenticación y autorización:** centralizar la validación de credenciales (tokens) antes de permitir el acceso a los servicios.
 - **Agregación:** componer respuestas a partir de la información de varios microservicios.
 - **Seguridad:** ofrecer una capa de protección que oculta la estructura interna de los microservicios.

2.2.3. Microservicios internos

Cada microservicio es un componente independiente con una responsabilidad única.

- **IntegrationService:** gestiona la comunicación con APIs de calendarios externos. Se encarga de importar eventos y calendarios desde servicios como **Google Calendar** o **Teamup**, manejando la autenticación y la transformación de datos necesaria.
- **EventManagerService:** contiene la lógica de negocio para la gestión de eventos (crear, leer, actualizar, eliminar - CRUD). También gestiona los contenidos asociados a un evento (imágenes, adjuntos) y dispara el evento que inicia el proceso de notificación (`I_NotificationTrigger`) cuando se produce un comentario. La decisión de manejar los comentarios dentro de `EventManagerService` viene dada por los siguientes motivos:
 - **Concepto de agregado (Aggregate):** en nuestro dominio, un comentario no tiene sentido por sí solo; existe únicamente en el contexto de un evento. El evento es la raíz de un agregado. Esto significa que el ciclo de vida del comentario (creación, borrado) está completamente gobernado por el evento al que pertenece. Por lo tanto, el microservicio que gestiona los eventos (`EventManagerService`) debe ser, por definición, el responsable de gestionar también sus comentarios.
 - **Separación de responsabilidades:** una vez que `EventManagerService` gestiona la creación del comentario, no se encarga de enviarlo. Delega esa responsabilidad disparando una notificación a través de la interfaz `I_NotificationTrigger`. El `NotificationService` la recoge y actúa.
 - **Consistencia de los datos:** al mantener la lógica de eventos y comentarios en el mismo servicio, se garantiza la consistencia transaccional. Cuando se añade un comentario, es una única operación atómica dentro del `EventManagerService`: se añade el comentario al evento y se actualiza el estado del evento si fuera

necesario. Si estuvieran en servicios separados, habría que gestionar transacciones distribuidas, lo cual añade más complejidad.

- **CalendarManagementService:** similar al anterior, pero centrado en la gestión de calendarios. Maneja la creación, edición y eliminación de calendarios, así como sus propiedades (título, organizador) y su estructura jerárquica.
- **SearchService:** proporciona la funcionalidad de búsqueda sobre los calendarios y eventos. Indexa la información relevante para permitir búsquedas eficientes por título, organizador, palabras clave, etc.
- **UserService:** es el responsable de la gestión de usuarios. Aunque la autenticación se delega a proveedores externos, este servicio almacena la información básica del perfil de usuario en la base de datos local y, fundamentalmente, gestiona los permisos y roles. Los demás servicios le consultan (`I_UserAccess`) para verificar si un usuario tiene autorización para realizar una determinada acción.
- **NotificationService:** orquesta el envío de notificaciones. Recibe el aviso del `EventManagerService` y, según la configuración del usuario, decide si la notificación debe enviarse por email (invocando a la Email Service API) o si debe almacenarse para mostrarse en el próximo inicio de sesión.

Se podría argumentar que `EventManager` y `CalendarManagement` podrían estar en un único "**servicio de contenido principal**". Sin embargo, mantenerlos **separados** es una mejor decisión a largo plazo. La gestión de calendarios (entidades de larga duración con jerarquías) y la de eventos (entidades más dinámicas y frecuentes) tienen lógicas y patrones de acceso distintos. La separación actual es más **limpia y robusta**.

2.2.4. APIs externas

Son servicios de terceros que dotan a la aplicación de funcionalidades clave sin necesidad de desarrollarlas desde cero.

- **Identity Provider (`I_TokenValidation`):** usa la librería `authlib` para OAuth 2.0. El componente gestiona la comunicación con proveedores externos (Google, Facebook) para validar la identidad de un usuario. La interfaz define un contrato único, como `validate(token)`, para que la aplicación verifique dicho token de forma segura sin importar el proveedor.
- **Google Calendar API (`I_GoogleCalendarAPI`):** usa la librería oficial `google-api-python-client`. El componente se conecta a la API de Google para leer y sincronizar eventos del calendario del usuario. La interfaz simplifica esta conexión, ofreciendo métodos claros como `get_events()` para que el resto de tu aplicación los consuma fácilmente.
- **Teamup API (`I_TeamupHTTPClient`):** usa la librería `requests` para hacer llamadas HTTP directas. El componente se encarga de la comunicación específica con la API de Teamup para importar eventos y calendarios. La interfaz aísla estas llamadas HTTP, protegiendo al resto del código de los detalles de esa implementación.
- **Cloud Storage API (`I_GoogleCloudStorageClient`):** usa la librería `google-cloud-storage`. El componente maneja la subida, descarga y gestión de archivos (imágenes, documentos) en un servicio de almacenamiento en la nube. La interfaz ofrece

métodos genéricos como `upload_file()`, para que tu aplicación no dependa de un proveedor de almacenamiento específico.

- **Maps API (`I_GoogleMapsClient`):** usa la librería `googlemaps`. El componente realiza tareas como la geocodificación (convertir direcciones en coordenadas) usando la plataforma de Google Maps. La interfaz proporciona funciones sencillas como `get_coordinates(address)` para añadir datos de localización a los eventos.
- **Translation API (`I_GoogleTranslationAPI`):** usa la librería `google-cloud-translate`. El componente se conecta al servicio de traducción para convertir textos a diferentes idiomas. La interfaz define un método estándar como `translate(text)` para preparar la aplicación para futuras funcionalidades multilingües.
- **Email Service API (`I_SendGridClient`):** usa la librería `sendgrid`. El componente gestiona el envío de correos transaccionales (notificaciones, recordatorios) a través de un servicio especializado. La interfaz define un contrato genérico como `send_email()`, lo que permite cambiar de proveedor de email en el futuro sin alterar el código principal.

2.2.5. Base de datos

- **PersistenceService (MongoDB, RNF1):** este componente actúa como la **capa de acceso a datos (DAL)** de la aplicación, implementando el **patrón de repositorio**. Su única responsabilidad es gestionar la persistencia y recuperación de datos, abstrayendo por completo la tecnología subyacente, que en este caso es MongoDB. Todos los microservicios acceden a esta capa exclusivamente a través de la interfaz `I_DataAccess`. Esta decisión de diseño es crucial por varias razones:
 - **Desacoplamiento:** la interfaz `I_DataAccess` actúa como un contrato que desvincula la lógica de negocio (microservicios) de los detalles de implementación del almacenamiento (de ahí que en el diagrama de componentes estén en paquetes distintos). Los servicios no saben que están hablando con MongoDB; solo saben cómo pedir o guardar datos.
 - **Intercambiabilidad:** si en el futuro se decidiera migrar de MongoDB a otra base de datos (como PostgreSQL o Firebase), solo sería necesario crear una nueva implementación del `PersistenceService` que cumpla con el contrato de `I_DataAccess`, sin modificar una sola línea de código en los microservicios.
 - **Centralización de la lógica de datos:** toda la lógica de consultas, optimizaciones y gestión de la conexión con la base de datos reside en un único lugar, lo que simplifica su mantenimiento y evolución.
 - **Testeabilidad:** este patrón facilita enormemente las pruebas unitarias. Se puede crear una implementación "mock" (falsa) de `I_DataAccess` para probar los microservicios de forma aislada, sin necesidad de levantar una instancia real de la base de datos.

3. Descripción de tecnologías elegidas

3.1. Backend (Python)

Para el desarrollo del backend, hemos seleccionado el lenguaje de programación **Python**.

Esta decisión se fundamenta en su **sintaxis clara y sencilla**, que disminuye su curva de aprendizaje y agiliza el proceso de desarrollo, y en la **tremebunda cantidad de soporte**, comunidad y librerías que tiene la tecnología. En el caso de querer integrar Inteligencia Artificial, también es el lenguaje con más facilidad para ello. Además, es un lenguaje con el que todos los integrantes del grupo han trabajado, por lo que la curva de aprendizaje y adaptación inicial tendrá aún menos pendiente.

3.1.1. Riesgos asociados

- **Rendimiento en tareas intensivas de CPU:** al ser un lenguaje interpretado, puede ser más lento que lenguajes compilados como Java en operaciones de cómputo intensivo.
- **Global Interpreter Lock (GIL):** puede limitar la concurrencia real en aplicaciones que dependen de múltiples hilos en una misma CPU, aunque en nuestro caso de estudio será posiblemente irrelevante debido a que la mayoría de tareas no requerirán concurrencia para cálculos intensivos.

3.1.2. Alternativas consideradas

- **JavaScript/TypeScript (Node.js):** se valoró por la posibilidad de usar un solo lenguaje en todo el stack (full-stack). Sin embargo, se descartó por la percepción de una menor robustez en su ecosistema para backend complejos en comparación con Python.
- **Java:** a pesar de su excelente rendimiento y robustez y de la experiencia que tenemos trabajando como equipo con esta tecnología, se descartó por su mayor complejidad y verbosidad, lo que podría ralentizar las fases iniciales del desarrollo.

3.2. Frontend (React + Vite)

En el frontend, nuestra elección es la librería **React** en combinación con la herramienta de construcción **Vite**.

Se ha optado por React debido a su arquitectura basada en componentes, que facilita la creación de interfaces de usuario dinámicas, reutilizables y fáciles de mantener, lo cual es ideal para una aplicación como *Basmati*. Su flexibilidad como librería nos permite tener un mayor control sobre la arquitectura de la aplicación.

La inclusión de Vite se justifica por su excepcional velocidad en el entorno de desarrollo, sus optimizaciones para producción y por su poca configuración inicial, lo que resulta en una experiencia de desarrollo superior y un mejor rendimiento para el usuario final. A todo esto, se le suma que el equipo ya ha trabajado con React en proyectos anteriores, por lo que el desarrollo debería ser, en teoría, más fluido que con otras tecnologías.

3.2.1. Riesgos asociados

- **Gestión del estado:** en aplicaciones grandes, la gestión del estado puede volverse compleja y requerir librerías adicionales como Redux, lo que añade una capa de aprendizaje.
- **Falta de unificación y "fatiga por decisión":** a diferencia de frameworks más opinados y completos como Angular, React es una librería que solo gestiona la capa de vista. Esto nos da libertad, pero también nos obliga a tomar decisiones sobre otros aspectos cruciales (enrutamiento, peticiones a la API, estructura de carpetas, etc.) que no incluye directamente la librería de React. Si el equipo no establece convenciones claras desde el principio, el código puede volverse inconsistente.

3.2.2. Alternativas consideradas

- **Angular:** se consideró por ser un framework completo y estructurado. Se descartó por su mayor rigidez y curva de aprendizaje más pronunciada, considerándolo menos ágil para este proyecto.
- **Vue.js:** si bien es conocido por su facilidad de uso, se eligió React por su ecosistema de librerías más grande y el mayor conocimiento sobre esta tecnología de los desarrolladores implicados.

3.3. API (FastAPI)

Para la implementación de la capa de API hemos elegido **FastAPI**, principalmente por su integración nativa con **Python**, lenguaje en el que desarrollaremos el backend. Su sintaxis declarativa y el uso de tipado estático opcional permiten crear endpoints de forma rápida y con un menor riesgo de errores. Además, genera automáticamente documentación interactiva (Swagger y ReDoc), lo que facilita tanto la validación de la API como la comunicación entre equipos.

FastAPI es altamente eficiente gracias a su soporte para **programación asíncrona (async/await)**, lo que lo convierte en una solución ideal para aplicaciones que requieren manejar múltiples peticiones concurrentes con bajo consumo de recursos. Su curva de aprendizaje es más sencilla que otros frameworks equivalentes, lo que agiliza el proceso de desarrollo y reduce la complejidad inicial para el equipo.

3.3.1. Riesgos asociados

- **Menor madurez:** a diferencia de frameworks más veteranos como Django REST Framework o Spring Boot, FastAPI es relativamente reciente y puede tener una comunidad y ecosistema más reducidos.
- **Curva de aprendizaje en asincronía:** aunque la sintaxis de FastAPI es sencilla, un mal manejo del paradigma asíncrono puede generar problemas de rendimiento y errores difíciles de depurar.
- **Dependencia de Python:** al estar fuertemente ligado al ecosistema de Python, podría limitar la portabilidad si en un futuro se decide migrar a un stack con otro lenguaje en el backend.

3.3.2. Alternativas consideradas

- **Django REST Framework (DRF):** ofrece un ecosistema maduro y completo para el desarrollo de APIs en Python, con gran cantidad de librerías y extensiones. Sin embargo, se descartó por ser más complejo y pesado de configurar para un proyecto que busca agilidad.

- **Spring Boot (Java):** framework muy robusto y ampliamente utilizado en entornos empresariales. Aun así, se descartó por la mayor complejidad del stack en comparación con FastAPI y por no estar alineado con la decisión de usar Python como lenguaje principal.
- **Express.js (Node.js):** muy popular y con una gran comunidad, además de la ventaja de unificar el stack con JavaScript. Sin embargo, se descartó porque el equipo tiene más experiencia en Python y se busca aprovechar su ecosistema, especialmente por la posible integración con herramientas de IA.

3.4. Base de datos (MongoDB)

Elegimos **MongoDB** porque su modelo de datos basado en documentos permite almacenar información estructurada y no estructurada de manera flexible, adaptándose fácilmente a los diferentes tipos de datos que maneja nuestra aplicación, como metadatos de usuarios, imágenes y configuraciones personalizadas.

MongoDB soporta **consultas avanzadas y agregaciones complejas**, lo que facilita realizar búsquedas, filtrados y análisis de datos directamente en la base de datos sin necesidad de lógica adicional en el backend.

3.4.1. Riesgos asociados

- **Escalabilidad y almacenamiento:** aunque MongoDB escala horizontalmente mediante sharding, configurarlo correctamente es complejo.
- **Backups y recuperación:** bases de datos NoSQL con grandes volúmenes de documentos binarios (como imágenes) pueden ser difíciles de respaldar y restaurar rápidamente.
- **Consistencia de los datos:** MongoDB es eventualmente consistente por defecto en entornos replicados, lo que significa que las réplicas pueden no estar inmediatamente sincronizadas. Esto puede provocar lecturas desactualizadas si no se gestionan correctamente las operaciones de escritura y lectura.

3.4.2. Alternativas consideradas

- **Firebase:** elegimos MongoDB en lugar de Firebase porque nos ofrece un modelo de datos más flexible, consultas avanzadas y mayor capacidad de escalabilidad real mediante sharding y replicación. Además, MongoDB evita la dependencia de un único proveedor, ya que puede desplegarse tanto en la nube como de forma autogestionada, garantizando mayor control sobre la infraestructura y los costes. Esto lo convierte en una solución más adecuada y sostenible que Firebase para nuestro proyecto. Firebase está más enfocada para aplicaciones en tiempo real, con limitaciones en consultas complejas, lo cual no casa en la aplicación que vamos a desarrollar.

3.5. Cloud (Google Cloud)

Para el despliegue en la nube hemos decidido usar **Google Cloud Platform (GCP)**. Esta decisión se fundamenta en su **sencillez de uso**, su **buena integración con tecnologías de datos e inteligencia artificial** y en las **ventajas económicas** que ofrece, como los créditos gratuitos para estudiantes y startups, lo que nos permite reducir significativamente los costes en la fase inicial del proyecto.

GCP proporciona una infraestructura altamente escalable, segura y con servicios gestionados que facilitan la operación, como balanceadores de carga, bases de datos y almacenamiento en la nube. Además, su interfaz resulta más intuitiva que la de otros proveedores y simplifica la configuración inicial de los servicios, lo que agiliza el desarrollo y despliegue de la aplicación.

3.5.1. Riesgos asociados

- **Menor cuota de mercado que AWS:** aunque GCP está creciendo rápidamente, su comunidad y cantidad de documentación son más reducidas en comparación con AWS.
- **Dependencia de servicios propios:** el uso de herramientas específicas de Google (BigQuery, Firestore, etc.) puede generar cierta dificultad en caso de migrar a otro proveedor.
- **Posibles costes a largo plazo:** aunque en un inicio es más económico por los créditos y tarifas, en escenarios de escalado muy grande los costes pueden equipararse a los de AWS.

3.5.2. Alternativas consideradas

- **Amazon Web Services (AWS):** se valoró por ser el proveedor más maduro, con el catálogo de servicios más amplio y con un ecosistema probado en entornos empresariales de gran escala. Sin embargo, se descartó debido a su mayor complejidad de configuración y a que sus precios iniciales resultan menos competitivos en comparación con Google Cloud, lo que lo hace menos atractivo para un proyecto académico y en fases iniciales.
- **Microsoft Azure:** se consideró por su integración con entornos empresariales y su buena oferta de servicios cloud, pero el equipo tiene más afinidad con las soluciones de Google y Python.
- **Heroku:** muy sencillo para proyectos pequeños, pero limitado en escalabilidad y con precios más elevados a medio plazo.

3.6. Sistema de almacenamiento de archivos (MongoDB + GCS)

Hemos decidido utilizar **MongoDB** como base de datos junto con el sistema de almacenamiento en la nube **Google Cloud Storage**. Los archivos de gran tamaño serán almacenados directamente en Google Cloud, mientras que en MongoDB únicamente se conservarán las URL de acceso correspondientes y los archivos más livianos.

Este enfoque garantiza un sistema **altamente escalable y seguro**, ya que separa la gestión de datos estructurados de la persistencia de archivos pesados. Aunque introduce dependencias externas adicionales, la arquitectura resultante permite un mejor rendimiento, reduce la carga de la base de datos y optimiza el uso de los recursos.

3.6.1. Riesgos asociados

- **Dependencia de terceros:** la arquitectura depende de la disponibilidad y estabilidad de Google Cloud Storage. Una caída del servicio o cambios en las condiciones de uso afectarían directamente la disponibilidad del sistema.
- **Costes variables:** el almacenamiento en la nube implica un modelo de pago por uso. El crecimiento exponencial en el volumen de archivos puede incrementar los costes de forma significativa si no se monitoriza ni se aplican políticas de optimización (compresión, almacenamiento en frío, políticas de expiración).
- **Seguridad y cumplimiento:** se deben configurar adecuadamente permisos, roles y accesos en Google Cloud para evitar fugas de información. Además, será necesario garantizar el

cumplimiento de normativas (p. ej., GDPR), dado que los datos estarán alojados en infraestructuras externas.

- **Complejidad operativa:** la integración entre MongoDB y Google Cloud Storage requiere una correcta sincronización entre URLs y archivos. Si se elimina un archivo en la nube sin actualizar la referencia en MongoDB (o viceversa), se podrían generar inconsistencias.

3.6.2. Alternativas consideradas

- **Otros servicios externos de almacenamiento (Dropbox, S3, etc.):** fueron considerados por su escalabilidad y optimización para servir archivos. Se descartaron porque añaden una dependencia externa aparte del propio cloud que ya usamos, y aumentan la complejidad de la arquitectura (gestión de APIs, control de permisos, sincronización con la base de datos...).
- **MongoDB GridFS,** fue una opción que barajamos debido a que simplifica la arquitectura en escenarios controlados, centraliza datos y archivos en un solo sistema y evita dependencias externas, lo que es especialmente útil en fases tempranas de un proyecto o en despliegues donde se prioriza la simplicidad sobre la escalabilidad extrema, sin embargo era demasiado poco escalable, ya que la recapturación de archivos como imágenes pesadas desde la base de datos, sería un proceso muy lento, debido a que GridFS guarda las imágenes en bytes y el proceso de transformarla de nuevo en un formato de imagen compatible, puede ser muy lento y costoso computacionalmente.